CSC-201: Computer Organization & Architecture

Autumn 2026

RISC201: Build, Run and Evaluation Guide

This document is a walkthrough. It starts from a fresh checkout, builds the project, runs a program, and then takes each part of the design in turn and shows the command that exercises it and the output that command produces. Every transcript below was captured from the program as it stands; none of it has been retyped or edited.

The sections after the first two are ordered to follow the assignment: the two pipelines, the microprogrammed control unit, the ALU and its arithmetic, the assembler and disassembler, the command-line debugger, the stack, and finally the measured comparison the evaluation asks for. Each one names the command, the thing to look at in the output, and what that figure demonstrates.

1. Build

Two commands. The first builds both binaries, the second runs every check in the project. There is nothing to configure and no dependency to install: the build needs `g++` with C++17 and nothing else.

```
make          # builds build/risc201asm and build/risc201sim
make test     # runs all eleven gates
```

The build is warning-clean under `-Wall -Wextra -Wpedantic -Wshadow -Wconversion`, so a silent build is itself a result. `make clean && make` rebuilds from nothing if you want to see that for yourself.

2. Check everything at once

`make test` runs eleven gates and stops at the first failure, so the last line is the verdict.

```
$ make test
test_alu: all checks passed
test_mdu: all checks passed
test_pipeline: all checks passed
test_control: all checks passed
test_fpu: all checks passed
test_stack: all checks passed
test_program_end: all checks passed
test_cli: all checks passed
  ok   all_instructions
  ok   errors (rejected as expected)
  ok   errors2 (rejected as expected)
  ok   fpu
  ok   macro_errors (rejected as expected)
  ok   macros
  ok   recursion
  ok   removed_bne (rejected as expected)
  ok   stack_errors (rejected as expected)
  ok   stack_errors2 (rejected as expected)
  ok   sum_array
  ok   vn_data
test_asm: all checks passed
test_evaluate: all checks passed
  ok   addition
  ok   function_call
  ok   recursion
  ok   multiply_divide
  ok   bit_reverse
  ok   quicksort
  ok   div_by_zero
  ok   stack_overflow
  ok   stack_underflow
  ok   misaligned
  ok   addition (.bin runs identically)
test_examples: all checks passed
```

The full suite. Each gate prints one line of its own; the assembler and the demonstration programs print one line per program.

What each gate establishes, and the command to run it alone:

Command	What it establishes
`make test-alu`	every ALU operation against the Section 2.3 table
`make test-mdu`	all four multiply and divide algorithms, iteration by iteration, including the worked divisions of Section 2.7
`make test-pipeline`	the cycle counts and per-hazard counters of both pipelines, and that the MDU latency constants match the iteration counts actually performed
`make test-control`	the control store against the worked words of Section 6.4, and that every control line is driven by exactly one pipeline stage
`make test-fpu`	every floating-point operation against the host's own binary32, bit for bit, over an edge list and a seeded random corpus
`make test-stack`	the three stack boundaries, each crossed by exactly one word
`make test-program-end`	that a program ends when the PC walks past its code, and that the data after it is never fetched
`make test-cli`	the debugger's commands, driven through piped input
`make test-asm`	assembles all twelve programs in `tests/asm/`, compares each image with the one recorded, and requires the six deliberately broken ones to be rejected
`make test-evaluate`	that the Section 5 sweep script runs end to end, and that it refuses to tabulate a program that faults
`make test-examples`	runs each of the ten demonstration programs and diffs its whole output against a recorded transcript

Two further targets are deliberately *not* part of `make test`, because they rewrite files that are committed: `make evaluate` runs the full timing sweep of Section 11, and `make examples` re-records the demonstration programs' images and transcripts.

3. Run a program end to end

The assembler takes one source file and writes the image beside it. The simulator loads that image and runs it. Nothing else is needed in between.

```
$ ./build/risc201asm examples/quicksort.asm
Wrote examples/quicksort.hex (72 words)
  code: 61 words from address 0x0
  data: 11 words from address 0xF4
```

The reply reports the code/data split, which the assembler writes into the image as a `; code <n>` header line. The simulator needs it because a program ends when the PC walks past the last word of code; without it the data would be fetched and executed.

The program sorts ten words in place. Reading that memory before and after the run is the whole demonstration:

```
$ printf 'mem 0xF4 10\nquit\n' | ./build/risc201sim examples/quicksort.hex
  0x000000F4:  0x0000001D   (29)
  0x000000F8:  0x00000007   (7)
  0x000000FC:  0x00000029   (41)
  0x00000100:  0x00000003   (3)
  0x00000104:  0x00000012   (18)
  0x00000108:  0x00000037   (55)
  0x0000010C:  0x0000000C   (12)
  0x00000110:  0x00000008   (8)
  0x00000114:  0x00000021   (33)
  0x00000118:  0x00000001   (1)
```

Before: the array as the assembler laid it down.

```
$ printf 'run\nmem 0xF4 10\nquit\n' | ./build/risc201sim examples/quicksort.hex
  0x000000F4:  0x00000001   (1)
  0x000000F8:  0x00000003   (3)
  0x000000FC:  0x00000007   (7)
  0x00000100:  0x00000008   (8)
  0x00000104:  0x0000000C   (12)
  0x00000108:  0x00000012   (18)
  0x0000010C:  0x0000001D   (29)
  0x00000110:  0x00000021   (33)
  0x00000114:  0x00000029   (41)
  0x00000118:  0x00000037   (55)
```

After: sorted. 0xF4 is where this program's data section begins, as the assembler reported above.

The same run on the 4-stage timing model reports what it cost:

```
$ printf 'pipe 4\nrun\nstats\nquit\n' | ./build/risc201sim examples/quicksort.hex
  Pipeline
------------------------------------------------------------
  Clock cycles      : 611
  Instructions      : 451
  MDU stalls        : 0
  FPU stalls        : 0
  Load-use stalls   : 0
  Branch-op stalls  : 46
  Squashed (bubbles): 102
------------------------------------------------------------
pipeline=4
mul_algo=booth
div_algo=non-restoring
cycles=611
instructions=451
mdu_stalls=0
fpu_stalls=0
load_use_stalls=0
branch_stalls=46
squashed=102
mdu_adder_ops=0
error=none
```

The boxed block is for a reader and the `key=value` block is for a script; they report the same run. `error=none` is the verdict. This matters because the simulator exits 0 even after a fault - a fault is a result, not a crash - so nothing in this project judges success by exit status.

4. The two pipelines, compared

Both designs execute the same instructions through the same semantics; only their timing differs. `pipe 4` and `pipe 6` select which model runs, and the same program under each is the comparison the assignment asks for.

```
printf 'pipe 4\nrun\nstats\nquit\n' | ./build/risc201sim examples/recursion.hex
printf 'pipe 6\nrun\nstats\nquit\n' | ./build/risc201sim examples/recursion.hex
```

```
pipeline=4
cycles=2799
instructions=2088
load_use_stalls=0
branch_stalls=265
squashed=355
error=none
--
pipeline=6
cycles=3153
instructions=2088
load_use_stalls=88
branch_stalls=441
squashed=443
error=none
--
```

Same 2088 instructions, 2799 cycles against 3153. The counters say why rather than leaving it to be guessed: the 6-stage pipeline records 88 load-use stalls where the 4-stage records none, because merging EX and MEM means a

loaded word is ready at the end of the same stage that computed its address. Its conditional branches also cost two bubbles instead of one, which is the 441 against 265. Section 11 runs this comparison across five programs.

5. The microprogrammed control unit

Execution is driven by a control store, not by a per-opcode switch statement. `ustore` with no argument prints the store itself: 64 entries, addressed by the opcode, each holding one 22-bit horizontal microword.

```
$ printf 'ustore\nquit\n' | ./build/risc201sim
  Control store - 64 entries, addressed by opcode
------------------------------------------------------------
  0x00  NOP     0x000000
  0x01  ADD     0x200000
  0x02  SUB     0x201000
  0x03  MUL     0x202002
  0x04  DIV     0x203002
  0x05  MOD     0x204002
```

The first six entries; the listing continues to opcode 0x24 and then names the reserved range.

Naming one instruction shows its microroutine: the control word projected onto the lines each stage consumes. The microPC is `opcode:step`.

```
$ printf 'ustore LD\nquit\n' | ./build/risc201sim
  Microroutine for 0x12 LD   (6-stage)
------------------------------------------------------------
  uPC    stage    microinstruction
  12:0   IF       -
  12:1   ID       -
  12:2   OF       -
  12:3   EX       ALUSrcB=imm  ALUOp=ADD
  12:4   MEM      MemRead=1
  12:5   WB       RegWrite=1  WBSrc=mem
  union of all steps = 0x220880  (Section 6.2 control word)
```

The union of the steps is the full control word, and the stage sets partition it, so every line is driven by exactly one stage - a line assigned to none would never reach the datapath, and one assigned to two would be driven twice. `make test-control` enforces that for both pipelines and checks this word, `0x220880`, against the worked example of specification Section 6.4.

During a run, `cycle` advances the clock one tick at a time and shows the microPC of whatever occupies each stage:

```
$ printf 'pipe 4\ncycle 3\nquit\n' | ./build/risc201sim examples/addition.hex
  cycle 1
   IF      0x0D ADDI            uPC=0D:0
   DE/OF   --
   EX/MEM  --
   WB      --
  cycle 2
   IF      0x0D ADDI            uPC=0D:0
   DE/OF   0x0D ADDI            uPC=0D:1
   EX/MEM  --
   WB      --
  cycle 3
   IF      0x01 ADD             uPC=01:0
   DE/OF   0x0D ADDI            uPC=0D:1
   EX/MEM  0x0D ADDI            uPC=0D:2  ALUSrcB=imm  ALUOp=ADD
```

By cycle 3 three instructions are in flight, each at its own step, and the stage that drives the ALU lines is the only one showing them. That is the microPC's step-by-step state the assignment asks the interface to display.

6. The ALU, the MDU and the FPU

Multiply and divide are not host operators. Four algorithms are built as the iteration the hardware performs, and `pipe` chooses which pair runs. Because each pair computes the same values, the figures that move are the cost of the arithmetic and nothing else.

```
pipe 4 booth         non-restoring
pipe 4 booth         restoring
pipe 4 shift-and-add non-restoring
pipe 4 shift-and-add restoring
```

```
mul_algo=booth  div_algo=non-restoring  cycles=207  mdu_stalls=189  mdu_adder_ops=133
mul_algo=booth  div_algo=restoring  cycles=282  mdu_stalls=264  mdu_adder_ops=209
mul_algo=shift-and-add  div_algo=non-restoring  cycles=207  mdu_stalls=189  mdu_adder_ops=146
mul_algo=shift-and-add  div_algo=restoring  cycles=282  mdu_stalls=264  mdu_adder_ops=222
```

The two multipliers both take 32 cycles, so they separate only in the adder column: 133 against 146 here. The two dividers separate in cycles instead, 207 against 282, because restoring pays one extra cycle per restore while non-restoring is a flat 33. `examples/multiply_divide.asm` is built to make each operand pattern attributable - measured one multiply at a time, `0x7FFFFFFF` costs Booth 2 adder operations against shift-and-add's 31, `0x55555555` costs it 32 against 16, and −1 is a tie at one each.

Which multiplier wins is therefore a property of the data, not of the algorithm. On the `arith` benchmark of Section 11 the order reverses: 1176 adder operations for Booth against 984 for shift-and-add.

Floating point is a separate unit, built from integer operations - unpack, align, operate, normalize, round once, pack - rather than wrapping the host's `float`. `freg` reads the register file as binary32 and shows the sticky IEEE exception register:

```
$ ./build/risc201asm tests/asm/fpu.s
$ printf 'pipe off\nrun\nfreg\nquit\n' | ./build/risc201sim tests/asm/fpu.hex
  Registers as binary32
------------------------------------------------------------
  R0  = 0x00000000 =              0    R1  = 0x00000084 = 1.84971397e-43
  R2  = 0x00000000 =              0    R3  = 0x42A00000 =             80
  R4  = 0x42100000 =             36    R5  = 0x40A00000 =              5
  R6  = 0x41800000 =             16    R7  = 0x00000010 = 2.24207754e-44
  R8  = 0x40200000 =            2.5    R9  = 0x42200000 =             40
  R10 = 0x41A00000 =             20    R11 = 0x00000001 = 1.40129846e-45
  R12 = 0x00000000 =              0    R13 = 0x00000000 =              0
  R14 = 0x00000000 =              0    R15 = 0x00000000 =              0
  FPSR  = izoux   (Invalid | divide-by-Zero | Overflow | Underflow | ineXact)
```

The lower-case letters in `FPSR` are the five IEEE conditions, none raised here. Floating-point exceptions accumulate rather than halting the machine, which is why `FDIV` by zero yields a signed infinity where integer `DIV` by zero stops the program. `make test-fpu` checks every operation against the host's binary32 bit for bit.

7. The assembler and its preprocessor

The assembler is two-pass: pass 1 assigns addresses and collects labels, pass 2 encodes. A preprocessor runs over the source before either pass, so neither pass learns that macros exist. One `SWAPW` line in `examples/quicksort.asm` is therefore four ordinary instructions by the time addresses are assigned, which the disassembly shows:

```
$ printf 'disasm 0x84 4\nquit\n' | ./build/risc201sim examples/quicksort.hex
   0x00000084:  0x4A140000   LD      R8, [R5 + 0]
   0x00000088:  0x4A580000   LD      R9, [R6 + 0]
   0x0000008C:  0x4E540000   ST      [R5 + 0], R9
   0x00000090:  0x4E180000   ST      [R6 + 0], R8
```

The expansion of `SWAPW R5, R6, R8, R9`: two loads and two stores, exchanging the words two pointers address.

Errors are reported with the line that caused them, and no output file is written at all if any pass reported one - a half-assembled image is never produced:

```
$ ./build/risc201asm tests/asm/errors.s
Error, line 2: unknown instruction 'FOO'
Error, line 3: unknown directive '.BOGUS'
Error, line 5: instruction 'ADD' is not allowed in the .data section
Error, line 6: .SPACE needs one count between 0 and 1048576
Error, line 9: label 'X' is defined more than once
Error, line 10: invalid label name '1BAD'
Error, line 11: missing operand (check the commas)
7 error(s), no output written.
```

`tests/asm/` holds twelve programs. Six must assemble and match a recorded image byte for byte; six are deliberately broken and must be rejected, three of them with their exact diagnostics checked. `make test-asm` is that gate.

8. The disassembler

Two commands share one disassembler and differ only in where the instruction word comes from. `disasm` reads words out of simulated memory, as used above. `decode` takes them as text, which is the requirement to reconstruct instructions from binary or hexadecimal word strings:

```
$ printf 'decode 0x34400003 60000008\ndecode @0x30 0x6C000008\nquit\n' | ./build/risc201sim
   0x00000000:  0x34400003   ADDI    R1, R0, #3  (0x3)
   0x00000004:  0x60000008   CALL    +8  -> 0x00000010
   0x00000030:  0x6C000008   BEQ     R0, R0, +8  -> 0x0000003C
```

It accepts a `0x` prefix, bare hex, a `0b` prefix or 32 bare binary digits, and resolves the ambiguity by shape rather than by guessing: 32 digits cannot be hex, and eight or fewer cannot be the binary spelling. `@0x30` lays the words out from that address instead of 0, which is what makes the branch target in the second line resolve correctly - a branch operand is an offset, so the mnemonic cannot be produced without knowing where the instruction sits.

9. The interactive debugger

The simulator is an interactive debugger with breakpoints, watchpoints, single stepping and state inspection. `help` lists the twenty-two commands and `help <command>` explains one; the companion CLI Command Guide documents each with a session of its own.

A watchpoint stops the machine after any instruction that changes the word it guards:

```
$ printf 'pipe off\nwatch 0xF4\nrun\nquit\n' | ./build/risc201sim examples/quicksort.hex
  Watchpoint #1 set on [0x000000F4]
  Stop reason     : Watchpoint
  Detail          : watchpoint #1 hit: [0xf4] = 0x1
```

It reports which watchpoint fired and the value written. Breakpoints and watchpoints are consulted by the functional model, so a session that uses them starts with `pipe off`; a pipeline run says so rather than appearing to have honoured one.

10. The stack: macros, safety and the visualiser

The stack is full-descending, and `LR` lives outside the register file, so a nested call has to save its return address with `PUSH LR` - the S-format L bit is its only path to memory. `examples/recursion.asm` computes fib(10) with two recursive calls per frame. Stopping it at its deepest point and asking for the stack renders the live frames:

```
$ printf 'pipe off\nbreak 0x40\nrun\nstack\nquit\n' | ./build/risc201sim examples/recursion.hex
  base 0x00010000   SP 0x0000FFB8   limit 0x00000068  (program image end)
  18 word(s) in use, 9 frame(s), 65360 byte(s) before overflow
                  +-------------------------------------+  base  frame 1
  0x0000FFFC      | 0x00000008  saved LR -> 0x00000008  |
  0x0000FFF8      | 0x0000000A  10                      |
                  +-------------------------------------+  frame 2
  0x0000FFF4      | 0x00000038  saved LR -> 0x00000038  |
  0x0000FFF0      | 0x00000009  9                       |
                  +-------------------------------------+  frame 3
```

The first frame of nine. The renderer cuts one frame from the next at each saved return address, which the simulator records as every `PUSH LR` executes - nothing in memory itself distinguishes a saved return address from a saved register.

Three boundaries are enforced in the memory stage, before the access, so a faulting instruction writes nothing and leaves SP where it was. Each has its own demonstration program, because a fault halts the machine and so only one can be shown per run:

```
$ for p in div_by_zero stack_overflow stack_underflow misaligned; do \
    printf 'pipe 4\nrun\nstats\nquit\n' | ./build/risc201sim examples/$p.hex \
      | sed -n 's/^error=//p'; done
div_by_zero: divide-by-zero exception at PC=0x00000010
stack_overflow: stack overflow at PC=0x00000004: PUSH to 0x0000000C is below the program image end 0x00000010
stack_underflow: stack underflow at PC=0x0000000C: POP with SP=0x00010000 at the stack base 0x00010000
misaligned: unaligned word read at address=0x2
```

The text names the fault, the PC of the instruction that caused it, and the offending address. These are the runtime exception framework: an exception is recorded rather than thrown, the machine halts, and `stats` carries the verdict in its `error=` key.

11. The measured evaluation

One command sweeps both pipelines over five benchmark programs, then re-runs the arithmetic-heavy one under each MDU algorithm and against the one-cycle floor. It reads the simulator's `stats` block rather than scraping output meant for a person, and it checks the `error=` key rather than the exit status.

The five programs live in `tests/bench/`. They are not applications: each one is written to put a single kind of pressure on the datapath, so that a difference between the two pipelines can be attributed to a named mechanism rather than guessed at from an average. Each file's header comment says which, and names the specification section that predicts the behaviour.

Program	What it puts under pressure
`loaduse`	Every loaded word feeds the instruction immediately after it. The 4-stage design pays nothing, because its merged EX/MEM stage has a loaded value ready exactly when an ALU result would be; the 6-stage design pays one stall per load. This is the program on which the two designs differ most.
`branchy`	A nested countdown, so nearly every conditional branch is taken. Isolates the taken-branch penalty: one bubble on the 4-stage design, two on the 6-stage one, because its comparator sits a stage later.
`arith`	A multiply, a divide and a remainder every iteration, so the MDU holds the execute stage for most of the run. The two multiplies pass their bit pattern as `rs2`, the operand Booth recodes - one long run of 1s against strict alternation - which is where the adder count varies most while the 32 cycles do not. This is the program the algorithm sweep re-runs.
`calls`	Recursive factorial, so the run is dominated by `CALL` and `RET` bubbles and by the stack traffic of saving a return address.
`mixed`	All four pressures in rough proportion: a load feeding its consumer, a branch either way, a call per iteration and one MDU operation inside it. It exists so the comparison is not read off one pathological shape.

A sixth file, `tests/bench/faulting.s`, is deliberately not a benchmark: it divides by zero, and `make test-evaluate` points the sweep at it to prove the runner refuses to tabulate figures from a run that faulted.

```
make evaluate
```

program	stages	multiplier	divider	cycles	instrs	MDU stalls	load-use	branch	adder ops	note
branchy	4	booth	non-restoring	1086	663	0	0	220	0	built
branchy	6	booth	non-restoring	1287	663	0	0	220	0	built
loaduse	4	booth	non-restoring	120	85	0	0	16	0	built
loaduse	6	booth	non-restoring	153	85	0	16	16	0	built
arith	4	booth	non-restoring	1652	113	1512	0	12	1176	built
arith	6	booth	non-restoring	1665	113	1512	0	12	1176	built

program	stages	multiplier	divider	cycles	instrs	MDU stalls	load-use	branch	adder ops	note
calls	4	booth	non-restoring	357	79	248	0	0	18	built
calls	6	booth	non-restoring	359	79	248	0	16	18	built
mixed	4	booth	non-restoring	369	78	248	0	8	16	built
mixed	6	booth	non-restoring	386	78	248	8	8	16	built
arith	4	shift-and-add	non-restoring	1652	113	1512	0	12	984	multiplier sweep
arith	4	booth	restoring	2190	113	2050	0	12	1726	divider sweep
arith	4	default	non-restoring	908	113	768	0	12	780	bound: host multiplier
arith	4	default	default	140	113	0	0	12	0	bound: host multiplier and divider
arith	6	shift-and-add	non-restoring	1665	113	1512	0	12	984	multiplier sweep
arith	6	booth	restoring	2203	113	2050	0	12	1726	divider sweep
arith	6	default	non-restoring	921	113	768	0	12	780	bound: host multiplier
arith	6	default	default	153	113	0	0	12	0	bound: host multiplier and divider

The upper rows are the requirement 1a comparison. The 4-stage design takes fewer cycles on every program, by a margin that depends entirely on what the program does: the 6-stage pipeline needs 27 per cent more cycles on `loaduse`, which is nothing but load-use dependences, and under one per cent more on `arith`, where 1512 of its 1665 cycles are MDU stalls that both designs pay identically. The two designs are therefore not ordered. Cycles are only half of the question, because the two do not share a clock: the 6-stage pipeline is faster overall only where splitting the stages buys more frequency than this deficit costs, which on `loaduse` means a clock period at least 21.6 per cent shorter and on `arith` only 0.8 per cent. Specification Section 5.1 works that break-even out per program.

The lower rows are the arithmetic comparison on the one benchmark the MDU dominates. The multiplier sweep moves the adder column and not the cycles; the divider sweep moves both; and the `default` rows are the floor, where the operation is handed to the host in one cycle. Every row is a measurement the simulator actually ran. The table is also written to `build/evaluation.tsv` for further analysis.

12. The demonstration programs

`examples/` holds ten programs, one for each thing worth showing, each with its source, both assembled images and a recorded transcript of its own output. `examples/README.md` describes each one and what to look at.

Program	Demonstrates
`addition`	the shortest whole path; the only program with no data section, so its hex and raw images behave identically
`function_call`	the calling convention, and why a non-leaf must save LR
`recursion`	recursive Fibonacci with two calls per frame; nine live stack frames
`multiply_divide`	the four MDU algorithms, with the operand patterns that separate them
`bit_reverse`	two algorithms for one answer, checked against each other over all 32 single-bit inputs
`quicksort`	recursion, pointer partitioning and macro preprocessing; checks its own output
`div_by_zero`	the divide-by-zero exception, and that the faulting instruction writes nothing
`stack_overflow`, `stack_underflow`	the two stack boundary checks
`misaligned`	the unaligned-access exception

Any one of them can be checked on its own:

```
PROGRAMS=recursion sh tools/examples.sh
```

13. Reading a failure

Every gate reports in the same shape and stops at the first failure, naming the program and the reason:

```
FAIL quicksort: the hex image differs from the committed one
FAIL div_by_zero: expected a runtime fault, got error=none
FAIL recursion: output differs from the recorded transcript
```

A transcript failure is followed by a unified diff showing the line that changed. Because the images are checked before the transcripts, a deliberate change to a program needs `make examples` to re-record before the behavioural check is reached - and the diff should be read before that is committed, since re-recording is also how a broken program would be blessed.

The gates have themselves been checked by breaking each program on purpose and confirming that the expected gate fails: a flipped digit in a committed image, an inverted comparison in the sort, a divide-by-zero program that no longer divides by zero, a truncated raw image, a deleted transcript line, and a corrupted mask in the bit reversal. A gate that has only ever been seen to pass is not yet known to be a gate.